1. Problem Title & Source

Title: **Maximum Continuous Horse Betting Sequence**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

At a horse racing event, a gambler places bets on a sequence of horses.

Each bet is represented as a number in an array:

1 → Winning bet

0 → Losing bet

The gambler wants to analyze his betting pattern.

Your task is to **find the length of the longest continuous sequence of winning bets (1s)**.

In simple terms:

- You are given a binary array containing only **0s and 1s**.
- You must determine the **maximum number of consecutive 1s**.

3. Examples

Example 1

Input

6

1 1 0 1 1 1

Output

3

Explanation

Continuous sequences of 1s:

1 1 → length = 2

1 1 1 → length = 3

Maximum sequence:

3

Example 2

Input

8

1 0 1 1 1 0 1 1

Output

3

Explanation

Sequences of 1s:

1 → 1

1 1 1 → 3

1 1 → 2

Maximum sequence = **3**

4. Constraints

$1 \le N \le 100$

arr[i] ∈ {0,1}

Observations

- Only two values exist
- We just track **continuous sequences**

5. Core Concept (Pattern)

Array Traversal + Streak Counting

Key idea:

Track current streak of 1s

6. Thought Process

Step 1 — Maintain Two Variables

currentStreak

maxStreak

Step 2 — Traverse Array

If element = 1

currentStreak++

If element = 0

reset currentStreak = 0

Step 3 — Update Maximum

After each step

maxStreak = max(maxStreak, currentStreak)

7. Visualization

Example

[1,1,0,1,1,1]

Process

Element	Current Streak	Max
1	1	1
1	2	2
0	0	2
1	1	2
1	2	2
1	3	3

Final answer

3

8. Pseudocode

```
read n
read array

current = 0
maxStreak = 0

for each element

   if element == 1
      current++
   else
      current = 0

   maxStreak = max(maxStreak, current)

print maxStreak
```

9. Code Implementation

Python

```
n = int(input())
arr = list(map(int,input().split()))

current = 0
max_streak = 0
```

```
for x in arr:

    if x == 1:
        current += 1
    else:
        current = 0

    max_streak = max(max_streak, current)

print(max_streak)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for(int i=0;i<n;i++)
            arr[i] = sc.nextInt();

        int current = 0;
        int maxStreak = 0;

        for(int x : arr){

            if(x == 1)
                current++;
            else
                current = 0;
```

```
            maxStreak = Math.max(maxStreak,current);
        }

        System.out.println(maxStreak);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

Single traversal.

Space Complexity

O(1)

Constant variables.

11. Common Mistakes

1. Not resetting streak after 0
2. Updating max before increment
3. Using nested loops unnecessarily
4. Missing last sequence update

12. Detailed Dry Run

Input

8

1 0 1 1 1 0 1 1

Element	Current	Max
1	1	1
0	0	1
1	1	1
1	2	2
1	3	3
0	0	3
1	1	3
1	2	3

Output

3

13. Real-Life Applications

This pattern appears in:

- Stock price growth streak
- Network uptime monitoring
- Sensor signal stability
- Sports winning streak analysis

14. Common Traps

1. Counting total 1s instead of consecutive
2. Forgetting reset condition
3. Using unnecessary extra arrays
4. Off-by-one errors

15. Related Problems

Consecutive sequence problems:

LC 485 Max Consecutive Ones

LC 1004 Max Consecutive Ones III

LC 487 Max Consecutive Ones II

16. Alternate Approaches

Approach	Complexity	Notes
Nested loops	$O(N^2)$	inefficient
Streak tracking	$O(N)$	optimal

17. Why This Solution Works

Each time we encounter **1**, the current winning streak grows.

Encountering **0 breaks the streak**, so we reset.

Tracking the longest streak gives the **maximum continuous winning sequence**.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Allow **one 0 to be flipped**
2. Longest sequence with **at most K zeros**

3. Count number of **continuous sequences**
4. Longest **alternating sequence**